INTRODUCTION

This document summarizes the software of "UTerraria", a 2D video game that runs on the MSPM0 Launchpad and is written in C++. It is based on Terraria, and therefore also allows the player to engage in combat and modify the game world. The game fulfills the requirements of the 319K final project, including interrupts, graphics, input, and sound. The Mob, Player, Chunk, Vector2d, and Inventory classes abstract the components of the game world, with the Engine namespace handling game logic and I/O.

DESIGN OBJECTIVE

The project's purpose is to apply embedded systems concepts into a standalone product that completes a task and is globally-aware. Information learned in the course must be incorporated into the game, including analog and digital signal processing, user interfacing, and circuit design.

As part of the project's evaluation, other students will vote on its enjoyability. Therefore, the project should also be entertaining to play, which requires smooth and detailed graphics, relevant audio, and an intuitive user interface.

This project was made to be a device where the user can play a clone of Terraria, a popular 2D video game where the player fights monsters, collects resources from the world, and builds structures. As a result of this inspiration, the project must implement world modification, player inventory, and combat.

Design Requirements

The project's hardware was required to run on the MSPM0 Launchpad, and include an LCD, 2 buttons, and a slide potentiometer, which should be read via the microcontroller's ADC. On the software side, the program must incorporate at least two interrupt handlers, output at least two distinct sounds, and have at least three images that move around the screen.

To make the project more globally accessible, it was required to include at least one language other than English. Based on a language selection screen at the beginning of the program, the user would only be shown text in their selected language.

Hardware Context

As shown below, the software runs on a M0+ CPU, which interfaces with the buttons and slide potentiometer via GPIO, the speaker via a DAC, and the LCD and SD card via an SPI controller. The project's I/O system must handle these signals to connect the peripherals to the internal game engine.

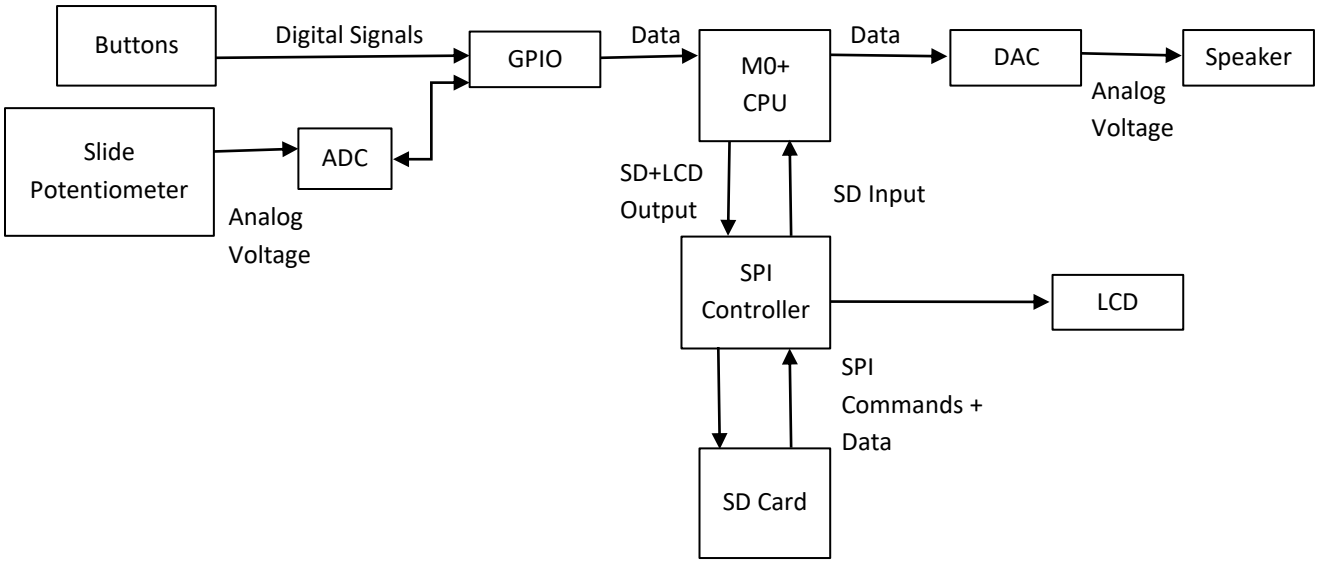

Figure 1: Hardware Overview

SOFTWARE DESIGN

The driver class reads in data from GPIO input, which it uses for player input. It then calls the game engine to progress the game state using the player's new actions. The engine calls the rendering subsystem based on the new game state, which outputs it to the LCD, and the engine also interfaces with the SD card for game saving/loading.

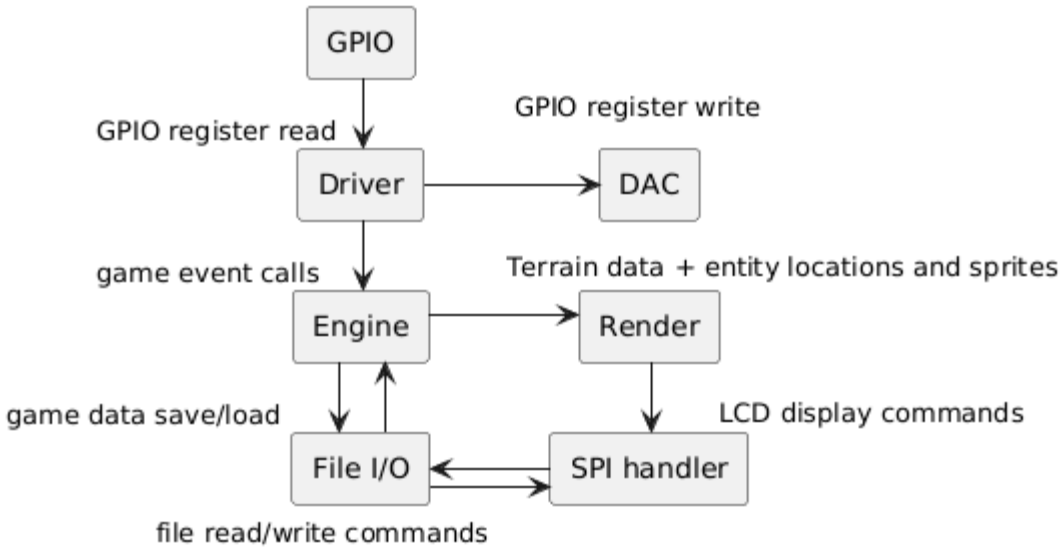

Figure 2: Software Overview

User Input

To allow the player to move their character throughout the world, four buttons can be pressed for moving up, down, left, and right respectively. Three other buttons are for breaking blocks, placing blocks, and entering a save menu. A slide potentiometer represents the player's inventory slot selector, so by moving the slide potentiometer the player could choose to place different blocks.

Buttons are read by the hardware as digital signal inputs, so they are abstracted in the program by the Button class. Each Button object is assigned a GPIO port and pin, and then will automatically be cast to the input value of that pin when used as an integer, allowing the program to treat Buttons as boolean values. For moving left, right, and down, and for placing and breaking blocks, the program will only check if the button is pressed down. However, for moving up (jumping), attacking, and accessing the save menu, the program checks for a rising edge in the button output, and so keeps track of if the button was pressed previously to determine when the button goes from unpressed to pressed.

The software receives an integer value representing the position of the potentiometer, which uses the known maximum value of the potentiometer to determine which ninth of the potentiometer's length it is at. It then selects the corresponding ninth of the player's inventory slots.

User Output

The internal game state is relayed to the player via visuals on an LCD and audio on a speaker. Having multiple mediums of output makes the game more immersive, and therefore more entertaining.

Display

A frame is output to the LCD every game tick, using TimerG12 interrupts for consistent frame times. The frame is centered on the player, so the player's location is used to determine the boundaries in the world of the screen, and from there the frame is rendered. To prevent flickering, the renderScreen() function renders all terrain and mobs at the same time, using the sprite of each mob to display it. There is also a rectangle at the top of the screen where the player's health bar and inventory are shown, so terrain will also not be rendered there. The selected slot in the player's inventory is outlined blue.

Sprites are internally represented as matrices, with each array inside representing a flattened matrix of an image, so instead of an image being a n x m matrix of pixel values, it is a single array of n*m length, which is the convention for representing images in embedded systems. The second dimension of the matrix is instead used to store multiple frames to produce animated sprites. Each sprite also has an alpha value matrix, with the same dimensions as the sprite but having a value of either 0 or 1. This allows the rendering engine to know which parts of a sprite are intended to be transparent, as represented by a 0 in the alpha value matrix, and will choose not to render that pixel of the sprite.

Multi-Language Support

All text is held in "Phrases", a 2D array of strings where the first index represents the language and the second represents where the text is displayed. For example,

Phrases[ENGLISH][GAME_OVER] holds the text that is displayed in English for the “game over” screen.

Audio

Sounds are currently played when the player successfully hits a Mob, while the player is walking, or when the player jumps. As opposed to the TimerG12 interrupts, the audio output uses SysTick interrupts, allowing audio output to occur independently of all other operations. The sounds were stored in 8-bit PCM format as arrays in ROM. Playing a sound arms SysTick interrupts and starts iterating across the sound’s array.

Game World

The device must hold a record of the internal game state, which includes the positions and conditions of the entities in the world and the world’s terrain, in order to determine the outcome of player inputs. This section will cover the internal representation of the game via various data structures, and then the engine responsible for progressing the game state.

Vector2d

The Vector2d struct is a pair of x and y coordinates, holding an integer for both axes. It overloads the operator for vector addition and subtraction, scalar multiplication, and includes a method distance(), which calculates taxicab distance (sum of distance on x axis and distance on y axis). All 2d vectors in the game, such as position and velocity, use this struct.

Mob

The Mob class holds a blueprint for all entities, containing the health, location, velocity, sprite, and direction the mob is facing. To know where the entity’s actions should have an effect, all Mobs have a direction that represents where it “intends” to make an action, which has a vertical and horizontal component. The vertical component can be either up, down, or neither, to represent that a Mob may not be aiming upwards or downwards, but the horizontal component must be either left or right. In memory, this is represented as a 3-bit value, with one bit for horizontal direction and two for vertical direction.

Mob objects are initialized with health, a sprite, and their starting position, with their starting direction initially set to looking right. The Orc, Guard, and StrongGuard subclasses only override this constructor with specific health values and sprites to increase the simplicity of adding the same type of Mob to the world.

The Player subclass has the additional attribute of an inventory, holding an array of 9 objects, each holding the item in that slot and the number of items. There is also an integer that keeps track of what inventory slot the player is currently on. The extra functionality allows the user's character to be represented as a Player.

Terrain

To facilitate future scalability, the terrain in the game world is comprised of Chunks. Each Chunk holds their position as recorded at their lower left corner, and a 32x32 matrix of integers representing the terrain data, as illustrated below. For reduced memory usage, each block in the world is represented by an 8-bit integer instead of an object, and any code that interacts with terrain data will have to know the mapping from integer values to block types.

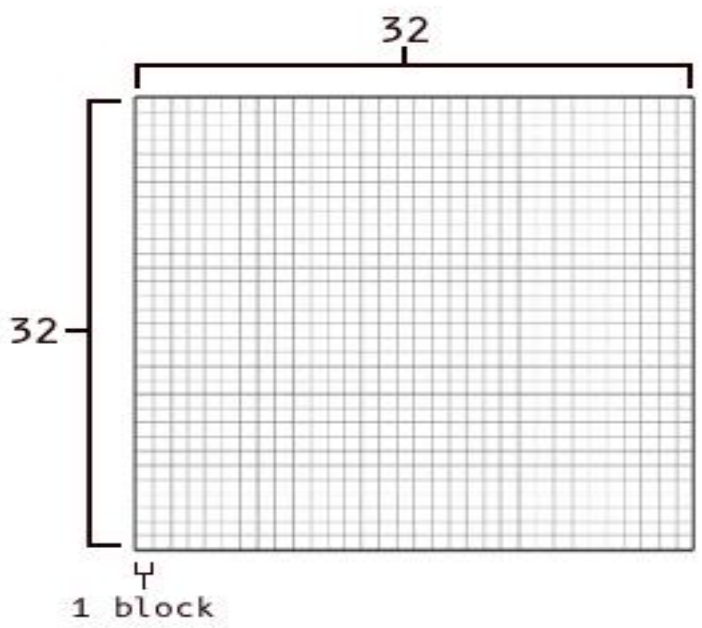

Figure 2: Block Matrix in Chunk

Since the game runs in memory, all world modifications will be reset after the game is restarted. The save menu allows players to save or load their world using either of four save slots. Using the FatFS library for file system interaction, terrain data is either saved to an SD card or loaded from it. Due to the internal representation of terrain data, the matrices holding the data can just be written or read as a stream of bytes, making the saving and loading process computationally simple.

Game Engine

The Engine namespace is the main handler of events in the game world, holding all terrain data and non-player Mobs. Its functions are called by the main method to alter the state of the game according to user input and other systems.

Game Data

The chunks[] array holds all the terrain data, containing a list of Chunks ordered left-to-right. By modifying this data structure, the engine can change the world's terrain. Currently, chunks[] has a length of 5, but can easily be increased to increase the world size. All non-Player Mobs are also held in a Mob array of length 10, so that the engine can update all mobs.

Game Events

Due to having access to all data, the engine is capable of handling any interaction between parts of the world. It uses entity and terrain data to simulate physics, uses entity data for the combat system, and uses player and terrain data for world modification.

Physics

The function updatePhysics() moves mobs according to their current position and velocity, adding the effect of gravity if they are off the ground. To prevent mobs from phasing through terrain, a collision box is made for the mob by checking the corners of its sprite and determining whether the new position of the mob would cause it to overlap with solid blocks in chunks[], in

which case it will be incrementally moved away from the overlapping region until no overlapping is detected.

Combat

The function followPlayer() moves hostile mobs closer to the player, changing the mob's direction and velocity to move toward the player's location. If its location is within 5 pixels of the player, it will attack the player, decreasing the player's health and starting a 20 tick cooldown until it can attack again.

The attack() function is for the player, which uses the player's direction to determine where their fist is, dealing damage to any mob that is close to the fist, and will also start an attack cooldown. Successfully attacking a mob will increase the player's score by the damage dealt to the mob.

Building

The function getBlock() takes an input position and determines the location in chunks[] representing the block at that position, returning an integer reference, which can be modified to modify that block in the terrain. When the player breaks a block, the player's direction will be used to determine where the player is breaking a block, and then if that block is not air (value in chunks[] is not 0), the block will become air and the item corresponding to the block will be added to the player's inventory. Placing a block will, if the block selected is air and the player has selected a placeable item, remove the item from the player's inventory and turn the block to it.

RESULTS

UTerraria fulfilled the design objective; however, there were several flaws that should be addressed in the future. Due to all blocks in the terrain having textures, the entire frame had to be rewritten every tick, resulting in 10 frames output per second, and increasing the SPI frequency to address this caused the SD connection to become unstable. Being constrained to the ARM M0+ is the greatest contributor to poor graphics performance, and significant improvements are likely not possible without proper graphics hardware.

No memory was manually deallocated by the software, so memory leaks are very likely to occur, which could cause early termination of the game as it runs out of memory. Proper memory management should be implemented in the future to address this issue.

Overall, the project accomplished the goal of applying embedded systems concepts to a larger real-world project.